

SmartIntern-Tracker — Upgrade Spec

Purpose: Add LinkedIn-style accounts + a personalized "jobs matching my CV" feed, plus supporting features. **How to use this doc:** Paste this whole file into Antigravity as your first message in a new task ("implement this spec in my current repo"), or feed it phase-by-phase. Database confirmed: **MongoDB**. Ask the agent to inspect the existing codebase first (models, whether it uses Motor/PyMongo/Mongoengine) and match that pattern rather than introducing a new one.

0. Ground truth about the existing repo (tell the agent this)

- Repo: `SmartIntern-Tracker` — Next.js (App Router) + TypeScript frontend; Python backend in `SmartIntern-backend/`.
 - **Auth already exists:** `POST /auth/signup` accepts `(email, password, full_name, branch, graduation_year, skills[])`. Do not rewrite this — extend it.
 - **AI matching already exists:** `SmartIntern-backend/api/ai_utils.py` has `(analyze_resume_match(resume_text, job_description))` — an async function that scores a resume against a single job description. Reuse this function; don't rebuild it. The new work is calling it at scale (many jobs per user) and persisting results instead of one-off calls.
 - Database is **MongoDB** (already set up and populated with at least the `User` collection from the working signup flow).
 - First instruction to give the agent: "Read `SmartIntern-backend/api/ai_utils.py`, the auth route file, and the existing MongoDB models/schema file before writing any code. Match the existing driver (Motor/PyMongo/Mongoengine) and naming conventions — don't introduce a second DB pattern."
-

1. Credentials & things YOU need to go get

Nothing below is optional-fluff — the agent cannot invent these for you. Get them first, put them in `SmartIntern-backend/.env`, and just tell the agent "these env vars exist, use `os.getenv()`." Database confirmed: **MongoDB** (already set up) — use Motor (async) or PyMongo, whichever the backend already uses; no new DB signup needed, just confirm the existing `MONGODB_URI` (or whatever it's currently named in `.env`) is the variable to reuse.

GEMINI_API_KEY — free, but read this first

Free, but Google tightened this in 2026: since April 1, 2026, Gemini Pro models are paid-

only — only Flash and Flash-Lite stay on the free tier (rate-limited, no card needed). Since matching already works in your repo, check which model string `ai_utils.py` calls:

1. Open `SmartIntern-backend/api/ai_utils.py`, find the `model=` string.
2. If it says something with "pro" in it (e.g. `gemini-2.5-pro`, `gemini-3-pro`) — either switch to `gemini-3-flash` or `gemini-2.5-flash` to stay free, or expect it to start billing your card.
3. To get/regenerate the key: go to **aistudio.google.com** → sign in with any Google account → **Get API key** → **Create API key** → copy it into `.env` as `GEMINI_API_KEY`. No card required for Flash-tier usage.

JWT_SECRET_KEY, JWT_ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES — free, no signup

You generate these yourself, nobody issues them:

1. Run `openssl rand -hex 32` in a terminal (Mac/Linux/WSL/Git Bash). No `openssl`? Run `python -c "import secrets; print(secrets.token_hex(32))"` instead.
2. Paste the output as `JWT_SECRET_KEY`.
3. Set `JWT_ALGORITHM=HS256` and `ACCESS_TOKEN_EXPIRE_MINUTES=60` (or whatever session length you want) — these are just config values, not real credentials.

Jobs data source — pick ONE, all free at this scale

Option	Free tier	Coverage
Adzuna	1,000 calls/month, no card	Best for UK/EU; India coverage exists but thinner
JSearch (via RapidAPI)	200 requests/month, no card	Wraps Google for Jobs — includes LinkedIn/Indeed/Glassdoor postings, better India coverage
Manual admin-added jobs	Unlimited, zero API cost	You add postings by hand — best for a demo/viva where you control exactly what's shown

Adzuna steps:

1. Go to `developer.adzuna.com/signup`, register with your email, verify it.
2. Your dashboard immediately shows `app_id` and `app_key` — copy both into `.env` as `ADZUNA_APP_ID` / `ADZUNA_APP_KEY`.

3. Test with: `https://api.adzuna.com/v1/api/jobs/in/search/1?app_id=...&app_key=...&what=software+intern`

JSearch steps (recommended if you want LinkedIn-style listings specifically):

1. Go to `rapidapi.com`, sign up free (email or GitHub).
2. Search "JSearch" in the marketplace, open it, subscribe to the **Basic (free)** plan — no card required.
3. Your `X-RapidAPI-Key` is shown on the app's dashboard page — copy into `.env` as `RAPIDAPI_KEY`.
4. Use the Playground on that page to test a query before wiring it into your backend.

Given the 200/month cap on JSearch, most student projects do better combining **manual admin-added jobs** as the main demo data, with Adzuna as a bonus live-data source — save JSearch for when you need LinkedIn-flavored results specifically.

RESEND_API_KEY (optional, Phase 7 — email alerts) — free

Free tier: 3,000 emails/month, 100/day, no card required.

1. Go to `resend.com`, sign up with email or GitHub.
2. You can send immediately from their shared test domain (`onboarding@resend.dev`) without owning a domain — fine for a student project.
3. Dashboard → **API Keys** → **Create API Key** → copy into `.env` as `RESEND_API_KEY`.

CLOUDINARY_URL (optional — resume file storage) — free

Free tier: no card required, 25 credits/month (plenty for storing PDFs at student-project scale).

1. Go to `cloudinary.com/users/register_free`, sign up (email, Google, or GitHub), verify email.
2. In the Console → **Settings** → **API Keys**, copy the **API Environment variable** — it's already formatted as `cloudinary://<api_key>:<api_secret>@<cloud_name>`.
3. Paste that whole string into `.env` as `CLOUDINARY_URL`.
4. Skip this entirely if you'd rather just store the resume as a blob/base64 in MongoDB — simpler, fine at this scale, just don't do it for large files.

Tell the agent explicitly: *"I will provide these env var names in `.env` — never hardcode keys, always read from environment."*

Give the agent this as the target schema; each of these is a collection, and the "(FK)" fields are just `ObjectId` references stored inside documents (embed vs. reference is the agent's call, but `UserJobMatch` should stay a separate collection since it grows unbounded):

User (*extend existing*) `(id, email, password_hash, full_name, branch, graduation_year, skills[], resume_id (FK), profile_complete_pct, created_at)`

Resume `(id, user_id (FK), raw_text, parsed_json (skills, experience, education – output of one Gemini parse call), file_url, uploaded_at)`

Job `(id, title, company, description, required_skills[], location, source ("adzuna" | "manual" | "jsearch"), external_id, posted_at, created_at)`

UserJobMatch `(id, user_id (FK), job_id (FK), match_score (0-100), matched_skills[], missing_skills[], computed_at)` — this is the table that turns single-shot matching into a ranked feed. One row per (user, job).

Application `(id, user_id (FK), job_id (FK), status (enum: wishlist|applied|oa|interview|offer|rejected), match_score_at_apply, applied_at, updated_at)`

SavedJob `(id, user_id (FK), job_id (FK), saved_at)`

Notification (*Phase 7*) `(id, user_id (FK), type, payload, read_bool, created_at)`

3. Backend API endpoints

Auth

Method	Path	Body	Notes
POST	<code>/auth/signup</code>	<i>(existing — unchanged)</i>	
POST	<code>/auth/login</code>	<code>{email, password}</code> → <code>{access_token}</code>	new
POST	<code>/auth/logout</code>	—	clears cookie
GET	<code>/auth/me</code>	— (auth required)	returns current user profile

Profile & Resume

Method	Path	Body	Notes
PUT	<code>/users/me</code>	profile fields	
POST	<code>/users/me/resume</code>	multipart file or raw text	parses via Gemini once, stores <code>parsed_json</code>
GET	<code>/users/me/resume</code>	—	

Jobs

Method	Path	Notes
GET	<code>/jobs</code>	list/browse all jobs (paginated)
POST	<code>/admin/jobs</code>	manual add (if not using an external API)
POST	<code>/jobs/sync</code>	pulls latest postings from Adzuna/JSearch into the <code>Job</code> table (cron-triggered or manual button)

Matching (the core new feature)

Method	Path	Notes
POST	<code>/jobs/match/run</code>	for the logged-in user (or all users, nightly): loop over jobs without a fresh <code>UserJobMatch</code> row, call <code>analyze_resume_match(resume.parsed_json, job.description)</code> , upsert score
GET	<code>/jobs/recommended?min_score=70</code>	returns jobs sorted by <code>match_score</code> desc for the logged-in user — this is what the dashboard calls

Applications

Method	Path	Notes
POST	<code>/applications</code>	<code>{job_id, status}</code>
GET	<code>/applications</code>	logged-in user's tracker board data
PATCH	<code>/applications/{id}</code>	update status (drag-drop in kanban)

4. Frontend pages/components

- `app/auth/login/page.tsx`, `app/auth/signup/page.tsx`
 - `context/AuthContext.tsx` — holds user + token, wraps app, redirects unauthenticated users away from `/dashboard`
 - `app/dashboard/page.tsx` — calls `/jobs/recommended`, renders `<JobCard>` list with a match-% badge
 - `app/profile/page.tsx` — edit profile + `<ResumeUploader>`
 - `app/applications/page.tsx` — `<KanbanBoard>` (columns = status enum)
 - `app/jobs/[id]/page.tsx` — job detail + matched/missing skills breakdown + "Apply" button
 - Shared components: `JobCard`, `MatchBadge` (color-coded % pill), `KanbanBoard`, `ResumeUploader`, `Navbar` (shows login state)
-

5. Build phases (give these to the agent one at a time, in order)

Phase 1 — Auth end-to-end

- ☐ Add `POST /auth/login` (verify password hash, issue JWT in httpOnly cookie)
- ☐ Add `AuthContext` + login/signup pages in frontend
- ☐ Add route protection middleware for `/dashboard`, `/profile`, `/applications`

Done when: you can sign up, log in, refresh the page, and stay logged in.

Phase 2 — Structured resume storage

- ☐ `POST /users/me/resume` — accepts resume text/file, calls a Gemini prompt once to extract `{skills, experience, education}` JSON, stores it

Done when: uploading a resume once produces a reusable structured profile, not raw text re-parsed every time.

Phase 3 — Jobs table + ingestion

- ☐ Create `Job` table/model
- ☐ Either: build `/jobs/sync` pulling from Adzuna/JSearch, OR build a simple `/admin/jobs`

form to hand-add postings

Done when: the `Job` table has at least 20-30 real rows to test matching against.

Phase 4 — Batch matching engine

- ☐ `POST /jobs/match/run` loops jobs × the user's `parsed_json` resume through `analyze_resume_match`, upserts into `UserJobMatch`
 - ☐ Add basic caching: skip jobs already scored for this resume version
- Done when:** running it once populates real scores you can query.

Phase 5 — Recommended feed UI

- ☐ `GET /jobs/recommended?min_score=`
- ☐ Dashboard page renders sorted `JobCard`s with `MatchBadge`

Done when: logging in shows only your top-matching roles, not the full job list. **This is your original ask — the milestone to demo.**

Phase 6 — Application tracker

- ☐ `Application` model + endpoints
- ☐ Kanban board UI, drag between statuses

Done when: clicking "Apply" on a job card creates a tracked application with the score locked in at that moment.

Phase 7 — Notifications / polish (optional, do last)

- ☐ Weekly digest email via Resend/SMTP: "N new roles matched you this week"
- ☐ Saved/bookmarked jobs
- ☐ Missing-skills report per job, rendered as a simple list

6. Instructions to paste alongside this doc

"Implement this in phases, one phase per task. Before writing code for a phase, inspect the current repo structure and existing files (models, routes, `ai_utils.py`) and tell me what already exists vs. what's new. Don't modify `/auth/signup` or `analyze_resume_match`'s signature — build around them. After each phase, give me a short summary of what changed and how to test it manually before moving to the next phase."